

Sistemas Operativos

Facultad de Informática
Universidad Complutense de Madrid

Módulo 3 - Procesos



Outline

1 Introducción

2 Multitarea

3 Información de un proceso

4 Formación y estados de un proceso

5 Procesos y ficheros

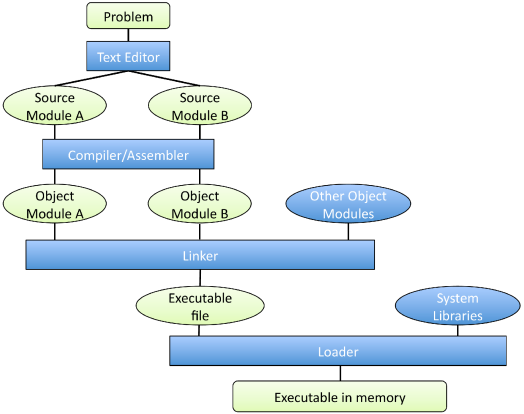
6 Señales

7 Hilos o threads

Concepto de Proceso

- **Programa**: fichero ejecutable en un dispositivo de almacenamiento permanente
- **Proceso**: Programa en ejecución
  - Conjunto de recursos que permiten la ejecución del programa
    - Mapa de memoria
    - Ficheros abiertos
    - Hilos de ejecución
    - ...
- Un proceso puede constar de uno o varios hilos de ejecución
  - Hilo: flujo de ejecución de instrucciones independiente
  - La unidad mínima planificable en una CPU

Preparación del código de un proceso





Recuerda: comandos útiles

- gcc
  - Compilador C de GNU
  - Realiza todas las etapas
  - `gcc --save-temps hello.c -o hello.o`
- ldd
  - Modo de uso: `ldd <fichero-ejecutable>`
  - Permite ver las librerías dinámicas con las que hemos enlazado
- `nm/objdump`
  - Permiten visualizar partes de un ejecutable

Entorno del proceso

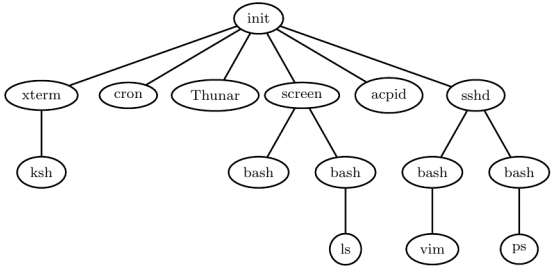
- **Entorno:** tabla que se pasa al proceso en su creación
  - Cada entrada es un par (*nombre*, *valor*)
  - Se puede consultar con el comando `env`
- El contenido inicial de la tabla se hereda del proceso padre
  - Puede cambiarse mediante:
    - Func. de la biblioteca estandar de "C": `getenv()`, `putenv()`, `unsetenv()`
    - Comandos del shell: `echo $NAME`, `export NAME=val`, `unset NAME`

Terminal #1

```
user@lab:~$ env
PATH=/usr/bin:/home/joe/bin
TERM=vt100
HOME=/home/joe
PWD=/home/joe/books/OperatingSystems
TIMEZONE=EDT
```

Jerarquía de procesos (UNIX)

- En UNIX los procesos se crean mediante clonación de un proceso existente
  - `fork()`
- Familia de procesos
  - Proceso padre
  - Proceso hijo



Consulta de procesos activos

- `ps`
  - Permite ver la información de todos los procesos en ejecución
  - `man ps` para consultar las múltiples opciones
- `top` (*table of processes*)
  - Muestra los procesos en ejecución, refrescando la información periódicamente
  - Permite interactuar con los procesos (p.ej., enviar señales)
- `pstree`
  - Permite visualizar el árbol de procesos

Usuarios y Grupos (I)

- Todo proceso del sistema tiene asociado un usuario propietario
- Usuario: Persona autorizada a utilizar un sistema
  - Se identifica en la autenticación mediante:
  - Nombre de usuario
  - Clave (password)
  - Internamente el SO le asigna un "UID" (user identification)
- Superusuario (*root*)
  - Tiene todos los derechos
  - Administra el sistema
- Grupo de usuarios
  - Los usuarios se organizan en grupos
  - Todo usuario ha de pertenecer al menos a un grupo
  - La existencia de grupos simplifica la administración del sistema

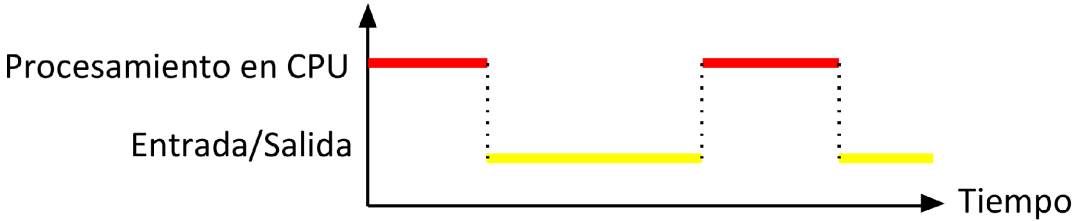
Usuarios y Grupos (II)

Identificadores de proceso y propietario (llamadas al sistema)

- `pid_t getpid(void)` / `pid_t getppid(void)`: identificador de proceso (PID) e identificador del proceso padre (PPID)
- `uid_t getuid(void)`/ `gid_t getgid(void)`: identificador real de usuario y grupo del propietario
- `uid_t geteuid(void)` / `gid_t getegid(void)`: identificador efectivo de usuario y grupo, usado por el SO para el control de acceso
- `int setuid(uid_t euid)` / `int setgid(gid_t egid)`: establece los identificadores efectivos
- Los ejecutables con los bits *setuid*/*setgid* activados fijan los identificadores efectivos a los del propietario del fichero Declaradas en `<unistd.h>`. Más información: `credentials(7)`

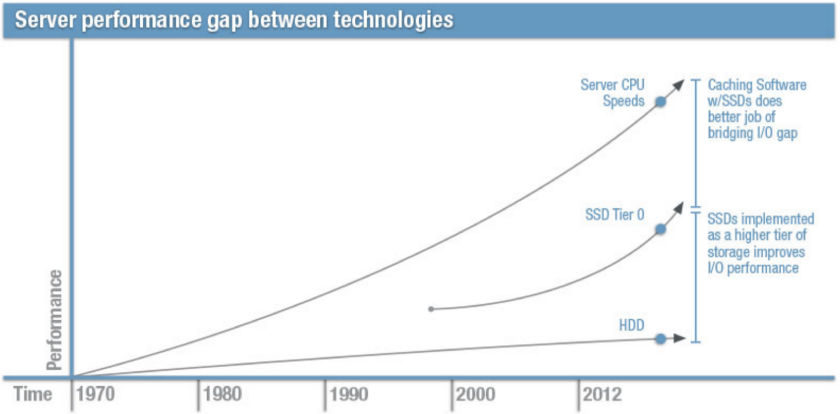
Motivación de la multitarea

- En los antiguos SSOO tipo batch (años 50/60) los procesos se ejecutaban en serie
  - Cola de procesos del sistema
- Cuando proceso en ejecución realiza operación de E/S la CPU se quedaba inactiva
  - Uso de E/S programada (espera activa)



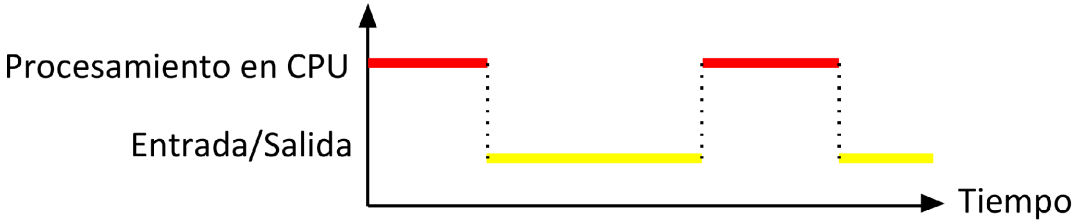
Motivación de la multitarea

- Desde la aparición de la tecnología CMOS hasta nuestros días, la CPU es cada vez más rápida que los dispositivos de E/S
  - E/S programada se vuelve más ineficiente



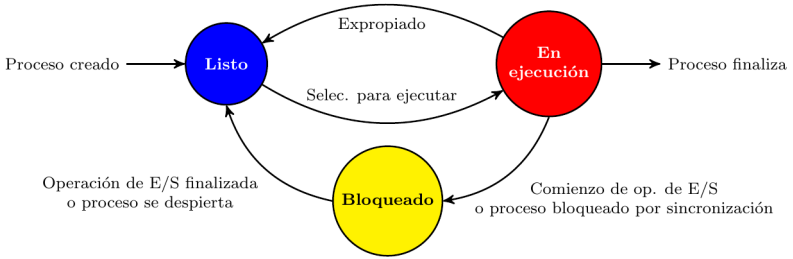
Base de la multitarea

- Explota el hecho de que los procesos alternan fases de E/S de procesamiento en CPU
- Paralelismo real entre E/S y CPU
  - Muchos procesos pueden realizar operaciones de E/S en paralelo
- Necesario almacenar en memoria la imagen de memoria de varios procesos



Estados básicos de un proceso

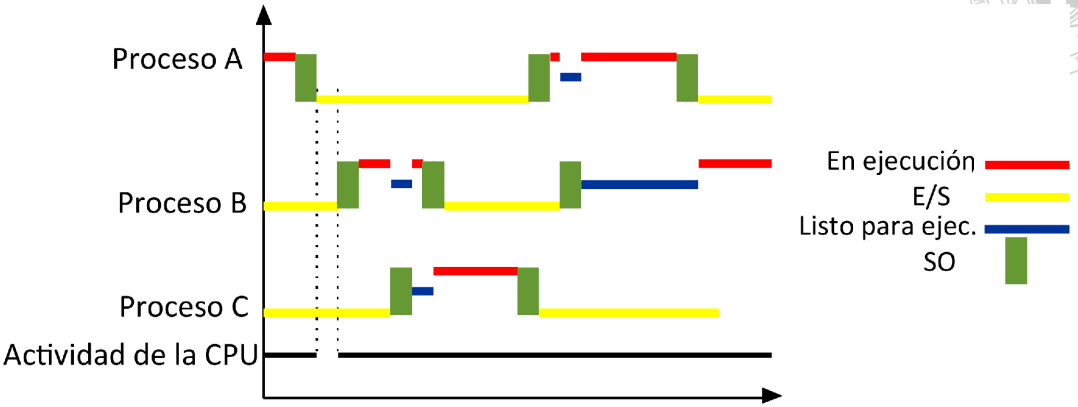
1. **En ejecución** (uno por procesador/core)
2. **Bloqueado** (en espera de completar E/S o por motivos de sincronización)
3. **Listo para ejecutar**



- Planificador: Componente del SO que decide qué proceso se ejecuta en cada procesador y en qué instante
- Proceso nulo o idle (uno por cada procesador)

Ejecución en un sistema multitarea

- Proceso nulo o idle

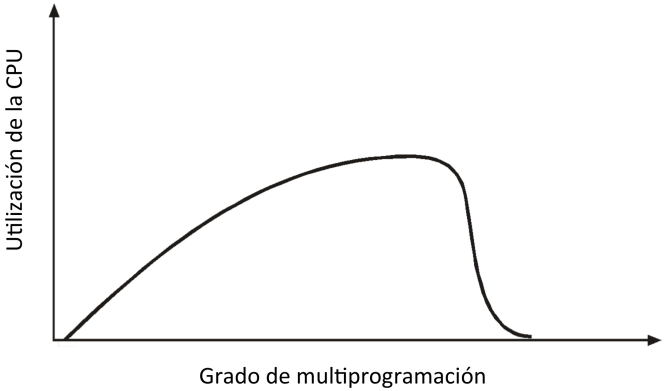


Ventajas de la multitarea

- Optimiza el uso de la CPU
- Permite el servicio interactivo simultáneo de varios usuarios de forma eficiente
- Facilita la programación, dividiendo los programas en procesos (modularidad)

Grado de multiprogramación

- **Grado de multiprogramación**: número máximo de procesos activos que un sistema puede gestionar de forma eficiente
- Para sistemas con memoria virtual:



Información de un proceso

- **Estado del procesador**: contenido de los registros del modelo de programación
- **Imagen de memoria**: contenido de los segmentos de memoria en los que reside el código y los datos del proceso
- **Bloque de control del proceso (BCP) o Descriptor de proceso**
  - Estado actual del proceso
  - Almacenamiento para el estado del procesador
  - Actualizado cuando proceso no se está ejecutando en la CPU
  - Identificadores pid, uid, etc.
  - Prioridad
  - Segmentos de memoria (espacio de direcciones)
  - Ficheros abiertos
  - Temporizadores
  - Señales
  - ...

Estado del procesador

- Está formado por el contenido de todos sus registros:
  - Registros generales
  - Contador de programa
  - Puntero de pila
  - Registro de estado
  - Registros especiales
- Cuando un proceso se está ejecutando su estado del procesador reside en los registros del computador
  - El estado del procesador de un proceso que no está en ejecución reside en el BCP

Imagen de memoria

- La imagen de memoria está formada por el conjunto de regiones de memoria que un proceso está autorizado a utilizar
  - Términos equivalentes: *espacio de direcciones*, *mapa de memoria*
- La imagen de memoria, dependiendo del computador, puede estar referida a memoria virtual o memoria física
- Si un proceso genera una dirección que está fuera del espacio de direcciones el HW genera una excepción que el SO captura
  - Típicamente, el SO mata el proceso cuando esto ocurre
  - Soporte HW –> *Memory Management Unit* (MMU)

Información del BCP

- **Información de identificación** :
  - PID del proceso, PID del padre (PPID)
  - ID de usuario y grupo reales (uid/gid reales)
  - ID de usuario y grupo efectivos (uid/gid efectivos)
- **Estado del procesador**
- **Información de control del proceso** :
  - Información de planificación y estado
  - Descripción de los segmentos de memoria del proceso
  - Recursos asignados (ficheros abiertos, ...)
  - Recursos de comunicación entre procesos
  - Punteros para estructurar los procesos en listas o colas
  - *task list* , árbol de procesos, *run queues* , *wait queues*

El sistema de ficheros /proc

- */proc* expone las estructuras internas del kernel como un árbol de directorios
- Cada PCB de proceso está disponible bajo un subdirectorio `/proc/<PID>`
- Los comandos `ps(1)` y `top(1)` se construyen sobre esta información

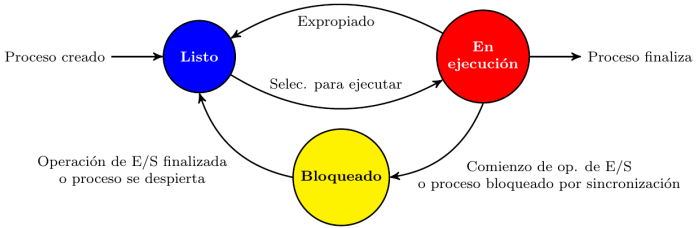
Terminal #1

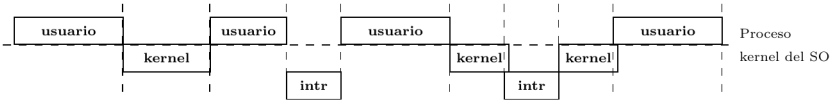
```
/proc/<PID>/
|-- cmdline
|-- cwd -> /home/user_OS
|-- environ
|-- exe -> /usr/bin/bash
|-- fd
|   |-- 0 -> /dev/pts/1
|   |-- 1 -> /dev/pts/1
|   |-- 2 -> /dev/pts/1
|   `-- 255 -> /dev/pts/1
|-- map_files
|   |-- 560f03ad6000-560f03b0d000 -> /usr/bin/bash
|   |-- 560f03b0d000-560f03bdf000 -> /usr/bin/bash
|   ...
|-- maps
|-- root -> /
|-- status
```

Contexto de un proceso

- Todo proceso mantiene un contexto de sistema de ficheros como parte de su PCB (`fs_struct` en Linux):
  - **Directorio de trabajo**: usado para resolver toda ruta relativa dentro del proceso
    - `char *getcwd(char *buffer, size_t size);`
    - `int chdir(const char *path);`
  - **Directorio raíz**: usado para interpretar rutas absolutas, restringiendo el espacio de nombres del sistema de ficheros del proceso (p. ej., contenedores)
    - `chroot(1,2)`

Estados del proceso



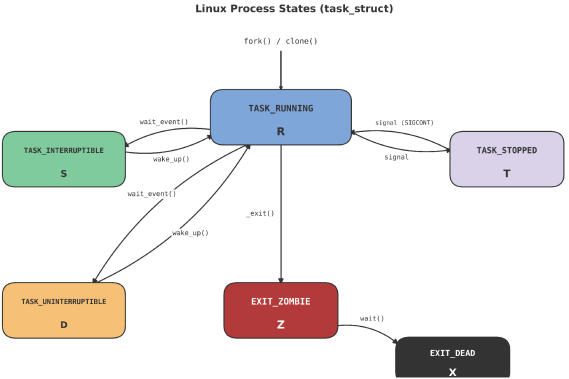


Estados del proceso en Linux

- El modelo abstracto Ready/Running/Waiting se traduce en los siguientes estados en el kernel de Linux:

Estado	Descripción
`TASK_RUNNING (R)`	Running (run queue, see Scheduling)
`TASK_INTERRUPTIBLE (S)`	Waiting I/O, interruptible by a signal
`TASK_UNINTERRUPTIBLE (D)`	Waiting for non-interruptible I/O
`TASK_STOPPED (T)`	Stopped by a signal
`EXIT_ZOMBIE (Z)`	Terminated process awaiting synchronization

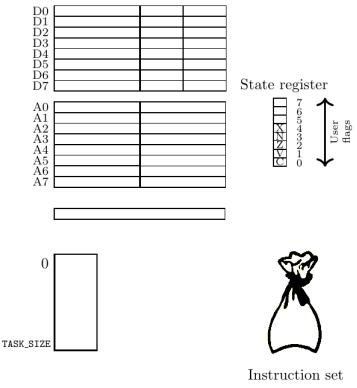
Estados del proceso



Cambio de modo del procesador

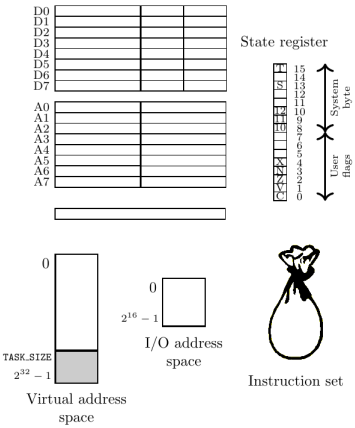
- Cuando se produce una interrupción o excepción mientras un proceso se ejecuta en modo usuario, el procesador cambia de modo de ejecución (a modo kernel)
- Al producirse la interrupción/excepción:
  - Se salva el valor de los registros (estado del procesador) en la pila de kernel del proceso
  - Se pasa a ejecutar la rutina de tratamiento de interrupción/excepción, bajo el control del SO
- Al finalizar la rutina, si el proceso actual sigue "en ejecución":
  - Restaura el estado de los registros del procesador
  - Termina con una instrucción RETI (retorno de interrupción). Dos acciones simultáneas
    1. Restituye el registro de estado (bit de nivel de ejecución)
    2. Restituye el contador de programa (para el nuevo proceso)

Modo usuario vs. modo kernel



- Modo Usuario

Modo usuario vs. modo kernel



- Modo Kernel

Cambio de contexto

- El cambio de contexto es el conjunto de acciones que realiza el SO para cambiar el proceso que está actualmente en ejecución en una CPU
- Acciones:
  1. Salvar el contexto del proceso saliente (registros del modelo de programación) en el BCP
  2. Cambiar el estado del proceso saliente (En ejecución -> Otro Estado)
  3. Intercambio de los espacios de direcciones
     - Tabla de páginas + Segmentos de memoria que puede usar un proceso
     - En x86. CR3 (Puntero al directorio de páginas) + LDTR (Dir. de Local Descriptor Table)
     - Invalidación de entradas de la TLB si lo exige la arquitectura
  4. Cambiar el estado del proceso entrante, (Listo -> En ejecución)
  5. Restaurar su contexto (BCP -> registros) y volver a modo usuario

Cambio de contexto

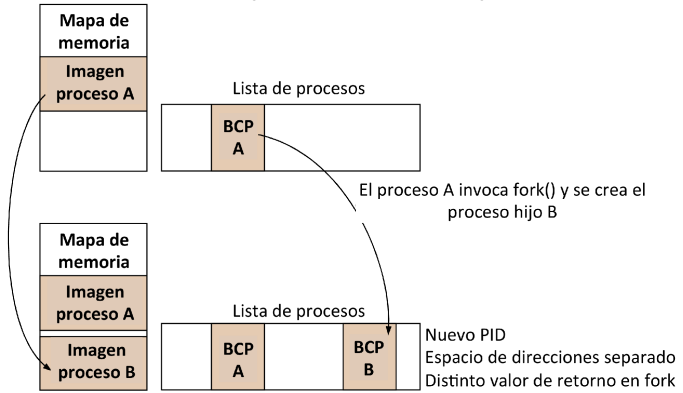
- Puede llegar a ser una operación bastante costosa
  - Reconstrucción del estado de la cache
  - Sobrecarga asociada a la activación del espacio de direcciones
- El cambio de modo de ejecución del procesador no siempre desencadena un cambio de contexto
  - Si el proceso actual sigue listo para ejecutar después de procesar la interrupción/excepción y el planificador no decide expropiar el proceso, no habrá cambio de contexto

Servicios de gestión de procesos (POSIX)

- Identificación de procesos
- Entorno de un proceso
- Creación de procesos
- Cambio del programa de un proceso
- Esperar la terminación de un proceso
- Finalizar la ejecución de un proceso
- Información sobre procesos

Servicios POSIX: `fork()`

- Crea un proceso clonando al padre



Servicios POSIX: `fork()`

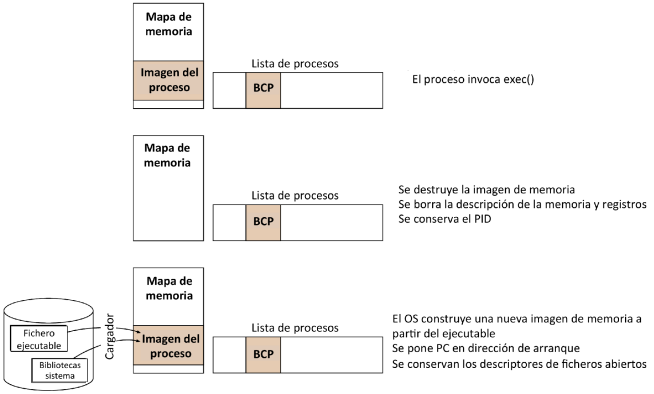
- Servicio: `pid_t fork(void);`
- Devuelve:
  - En caso de éxito retorna dos veces, una en el proceso hijo y otra en el padre
  - Retorna 0 en el hijo y el PID del hijo en el padre En caso de fallo retorna solo una vez (valor de retorno -1)
- Descripción ** Crea un proceso hijo que ejecuta el mismo programa que el padre
  - El proceso hijo sólo tiene un hilo
  - Hereda los ficheros abiertos (se copian los descriptores)

Servicios POSIX: fork()

```
void main()
{
    pid_t pid;
    ..
    pid = fork();
    if (pid == 0) {
        /* Proceso hijo */
        ...
    } else if (pid>0){
        /* Proceso padre */
        ...
    } else{
        /* Error */
    ...
    }
...
}
```

Servicios POSIX: `exec()`

- Cambia el programa de un proceso



Servicios POSIX: `exec()`

- Funciones exec(3), todas construidas sobre `execve(2)`:
  - `int execl(const char *path, const char *arg, ...);`
  - `int execv(const char *path, char *const argv[]);`
  - `int execlp(const char *file, const char *arg, ...);`
  - `int execvp(const char *file, char *const argv[]);`
  - `int execle(const char *path, const char *arg, ..., char *const envp[]);`
  - `int execve(const char *path, char *const argv[], char *const envp[]);`
  - `int execvpe(const char *file, char *const argv[], char *const envp[]);`
- Convención de nomenclatura:
  - **l**: los argumentos se pasan como una lista
  - **v**: los argumentos se pasan como un vector (array)
  - **p**: busca el ejecutable en PATH
  - **e**: permite al invocador especificar el entorno explícitamente

Servicios POSIX: `exec()`

- Funciones disponibles
  - `int execl(const char *path, const char *arg, ...);`
  - `int execlp(const char *file, const char *arg, ...);`
  - `int execvp(const char *file, char *const argv[]);`
- Argumentos:
  - `path, file`: nombre/ruta del ejecutable
  - `arg`: argumentos del programa
- Descripción
  - Devuelve -1 en caso de error, en caso contrario **no retorna**
  - El mismo proceso ejecuta otro programa
  - Los ficheros abiertos permanecen abiertos
  - Las señales con la acción por defecto seguirán por defecto, las señales con manejador tomarán la acción por defecto

Servicios POSIX: exec()

- Ejecutando `ls -l`
  - Ejecutable en `/bin`
  - Ejecutar comando "`which ls`" para obtener ruta completa
  - Asumimos que `/bin` está en el PATH (`echo $PATH`)

```
/* Primera alternativa */
execl("/bin/ls","/bin/ls","-l",NULL);

/* Segunda alternativa */
execlp("ls","ls","-l",NULL);

/* Tercera alternativa */
char* arguments[]={"ls","-l",NULL};
execvp(arguments[0],arguments);
```

Terminación de un proceso: exit()

- Servicio: `int exit(int status);`
- Argumentos:
  - Código de retorno para el proceso padre
- Descripción:
  - Finaliza la ejecución del proceso.
  - Se cierran todos los descriptores de ficheros abiertos
  - Se liberan todos los recursos del proceso

`_exit()` frente a `exit()`

- Un proceso puede terminar voluntariamente llamando directamente a `_exit(2)`, o volviendo de `main()` / llamando a `exit(3)`, que es un envoltorio ligero de libc sobre esta
- `void _exit(int status);`
  - `status` es el estado de salida; solo se conservan los 8 bits menos significativos (los valores deben ser < 255)
  - Por convención, `0` (`EXIT_SUCCESS`) significa éxito y `1` (`EXIT_FAILURE`) significa error
  - Nunca devuelve `errno` o `-1` como estado
  - Disponible para la shell a través de `$?`, o para el proceso padre a través de `wait(2)`
  - El proceso padre recibe la señal `SIGCHLD`
- `exit(3)`, antes de invocar a `_exit(2)`, además:
  - Llama a las funciones registradas con `atexit(3)` y `on_exit(3)`
  - Vacía y cierra los flujos `stdio(3)` abiertos
  - Elimina los ficheros temporales creados con `tmpfile(3)`

Espera terminación de proceso hijo: `wait()`

- Servicio

```
#include <sys/types.h>
pid_t wait(int *status);
```

- Argumentos:
  - `status`: parámetro de salida. Código de terminación del proceso hijo
- Descripción
  - Devuelve el identificador del proceso hijo o -1 en caso de error
  - Permite a un proceso padre esperar hasta que termine un proceso hijo. Devuelve el identificador del proceso hijo y el estado de terminación del mismo

Esperar a un hijo concreto: `waitpid()`

- Servicio: `pid_t waitpid(pid_t pid, int *wstatus, int options);`
- `pid` especifica a qué procesos hijo esperar

Valor	Significado
> 0	El proceso cuyo PID es `pid`
0	Cualquier proceso hijo del grupo de procesos del invocador
<-1	Cualquier proceso hijo (igual que `wait(2)`)
< -1	Un proceso hijo cuyo PGID es -pid

- `options` es un OR a nivel de bits de:
  - `WNOHANG`: retorna inmediatamente si ningún hijo ha terminado
  - `WUNTRACED`: también retorna si un proceso hijo se ha detenido
  - `WCONTINUED`: también retorna si un proceso hijo detenido se ha reanudado

Esperar a un hijo concreto: `waitpid()`

- Servicio; `pid_t waitpid(pid_t pid, int *wstatus, int options);`
- `wstatus` lleva información de estado:
  - `WIFEXITED(s)`: verdadero si el hijo terminó normalmente con _exit(2); en ese caso WEXITSTATUS(s) devuelve el estado de salida
  - `WIFSIGNALED(s)`: verdadero si el hijo terminó debido a una señal; en ese caso WTERMSIG(s) devuelve el número de señal

Ejemplo

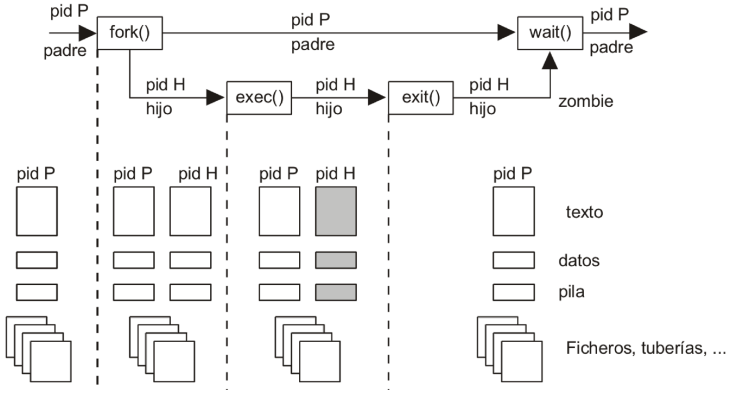
example.c

```
void main(){
    int a=0;
    int status;
    pid_t pid;

    pid = fork();
    if (pid == 0) {
    /* proceso hijo */
        a++;
    } else if (pid>0){
    /* proceso padre */
        wait(&status);
        a=a+2;
     } else{
        fprintf(stderr,"Can't fork()\n");
        exit(1);
     }
    printf("My PID is %d, a=%d\n",getpid(),a);
    exit(0);
}
```

- ¿Qué imprimirá por pantalla el proceso hijo con `printf()`?
- ¿y el padre?

Uso normal de los servicios



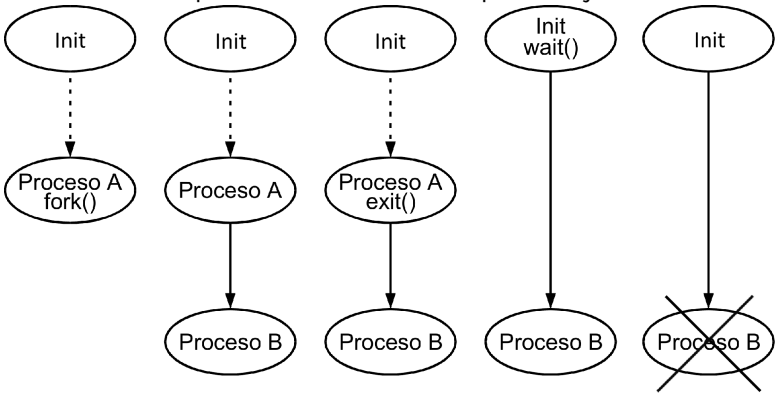
Programa de ejemplo

```c
#include <sys/types.h>
#include <stdio.h>

/* programa que ejecuta el comando 'ls -l' */
void main() {
    pid_t pid;
    int status;
    pid = fork();
    if (pid == 0) {  /* proceso hijo */
        execlp("ls","ls","-l",NULL);
        exit(1);
    } else { /* proceso padre */
        while (pid != wait(&status));
        exit(0);
    }
}
```

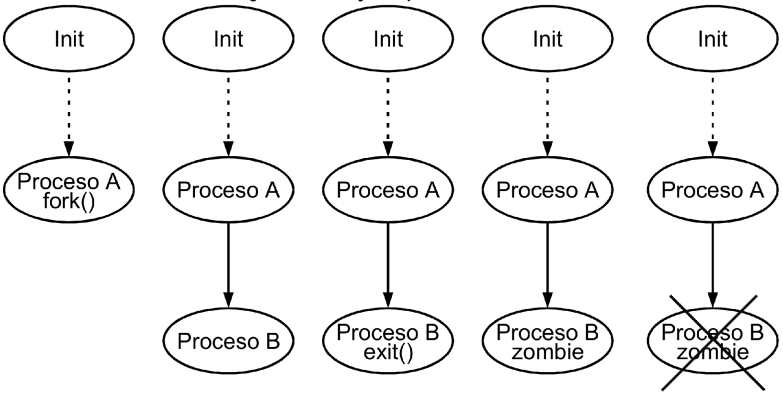
Ciclo de vida de un proceso (I)

- El padre muere –> INIT acepta los hijos



Ciclo de vida de un proceso (II)

- Zombie: el hijo muere y el padre no invoca `wait()`



Semántica de coutilización

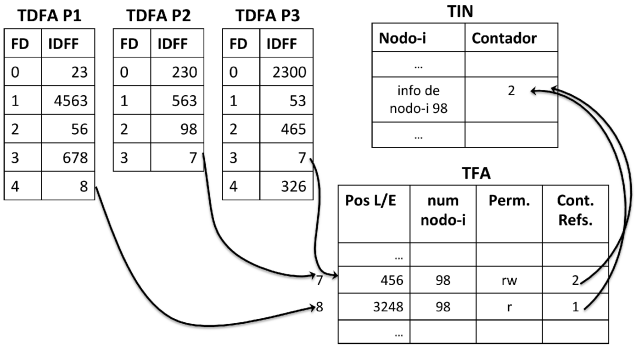
- Cualquier mecanismo de acceso a ficheros tiene problemas cuando varios usuarios trabajan con el fichero simultáneamente
- La semántica de coutilización especifica el comportamiento del acceso simultáneo a un fichero desde varios procesos
  - Conjunto de reglas que establecen también cuando los cambios realizados por un proceso en un fichero son visibles desde otro proceso diferente
- Tipos de semánticas
  - UNIX
  - Semántica de Sesión
  - Semántica de versiones
  - Ficheros inmutables

Semántica UNIX

- Cada fichero se expone como un "objeto global" a todos los procesos que acceden a él
  - Las escrituras son inmediatamente visibles para todos los procesos con el fichero abierto
  - El resultado de dos escrituras consecutivas a la misma región del fichero (desde varios procesos) es la segunda operación
- Los procesos emparentados comparten la visión lógica de los ficheros abiertos, incluyendo el puntero de posición
  - Ejemplo:
    1. El proceso P1 abre un fichero F y crea un proceso hijo P2 con `fork()`
    2. El proceso P2 hereda F: (a) puede acceder a F directamente y (b) cualquier cambio realizado en el puntero de posición realizado por P1 o P2 es visible desde cualquiera de los procesos
- La herencia de ficheros sólo afecta a los ficheros abiertos por P1 antes de invocar `fork()`

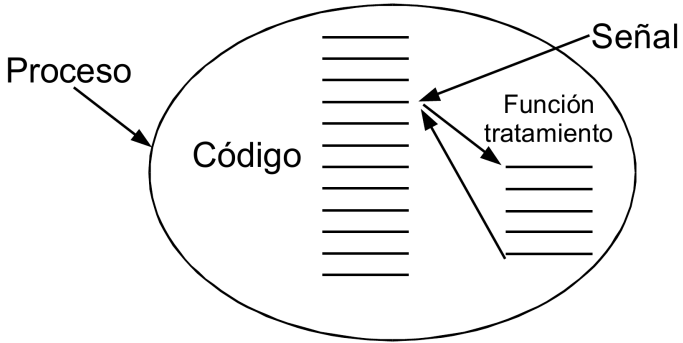
Tablas del Sist. de Gestión de Ficheros

- Tabla intermedia de nodos-i (**TIN**):Tabla global que incluye los nodos-i de los ficheros en uso
- Tabla intermedia de posiciones (**TFA**): Tabla global que incluye las distintas representaciones lógicas de los ficheros en uso
- Tabla de descriptores de ficheros abiertos (**TDFA**): Una por proceso



Concepto de señal

- Las señales son interrupciones al proceso
- Envío o generación:
  - Proceso –> Proceso (con mismo uid) con kill() o mediante el comando kill
  - SO –> Proceso



Señales

- Hay muchos tipos de señales, según su origen
  - `SIGILL` instrucción ilegal
  - `SIGALRM` vence el temporizador
  - `SIGKILL` mata al proceso
- El SO las transmite al proceso
  - El proceso debe estar preparado para recibirla
  - Especificando un manejador de señal con `sigaction()`
  - Enmascarando la señal con `sigprogmask()`
- Si no está preparado –> acción por defecto
  - El proceso, en general, muere
  - Hay algunas señales que se ignoran o tienen otro efecto
- `pause()`: para el proceso hasta que recibe una señal

Señales comunes

Señal	Descripción	Acción por defecto
SIGINT	Interrupción. Se puede generar con `Ctrl+C`	F
SIGTSTP	Parada. Se puede generar con `Ctrl+Z`	P
SIGSTOP	Detiene el proceso	P
SIGCONT	Reanuda un proceso detenido	–
SIGKILL (9)	Terminación abrupta	F
SIGSEGV	Violación de segmento	F, C
SIGTERM	Termina el proceso	F
SIGCHLD	Un proceso hijo ha terminado	I

- Leyenda: F = termina el proceso, P = detiene el proceso, C = además vuelca core, I = ignorada por defecto

Envío de señales: ejemplo

Terminal #1

```
user@lab:~$ sleep 50
^C
user@lab:~$ echo $$
5711
user@lab:~$ ^C
user@lab:~$ ^C
user@lab:~$
```

Terminal #1

```
user@lab:~$ ps -ef | grep sleep
user     5756  5711  0 09:19 pts/3    00:00:00 sleep 50
user     5758  5667  0 09:19 pts/2    00:00:00 grep sleep
user@lab:~$ kill -s SIGINT 5756
user@lab:~$ kill -s SIGINT 5711
user@lab:~$ kill -s SIGINT 5711
```

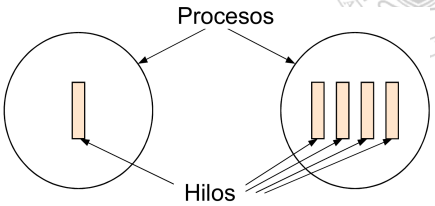
- La variable built-in $$ de BASH almacena el PID del proceso shell

Servicios POSIX para señales

- `int kill(pid_t pid, int sig);`
  - Envía la señal sig al proceso con `PID=pid`.
- `int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);`
  - Permite indicar qué acción se realizará (`act`) cuando el proceso recibe la señal signum. La acción previamente configurada se almacena en `oldact`
- `int pause(void);`
  - Bloquea el proceso hasta la recepción de una señal
- `unsigned int alarm(unsigned int seconds);`
  - Genera la recepción de la señal SIGALRM pasados seconds segundos
- `int sigprocmask(int how, const sigset_t *set, sigset_t *oset);`
  - Usado para examinar/modificar la máscara de señales del proceso

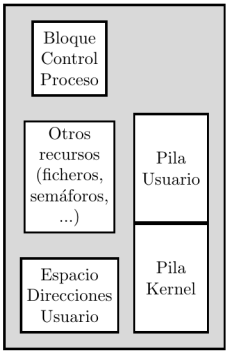
Hilos o threads

- Por Hilo
  - Contador de programa, Registros
  - Pila
  - Estado (ejecutando, listo o bloqueado)
  - Bloque de control de thread
- Por proceso
  - Espacio de direcciones de memoria
  - Variables globales
  - Ficheros abiertos
  - Procesos hijos
  - Temporizadores
  - Señales y semáforos

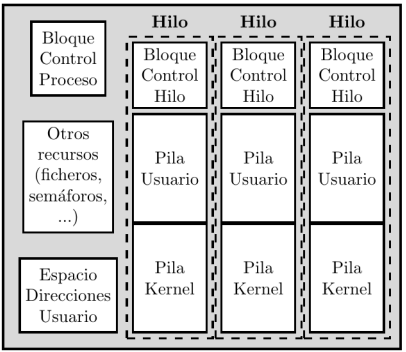


Mono-hilo vs Multi-hilo

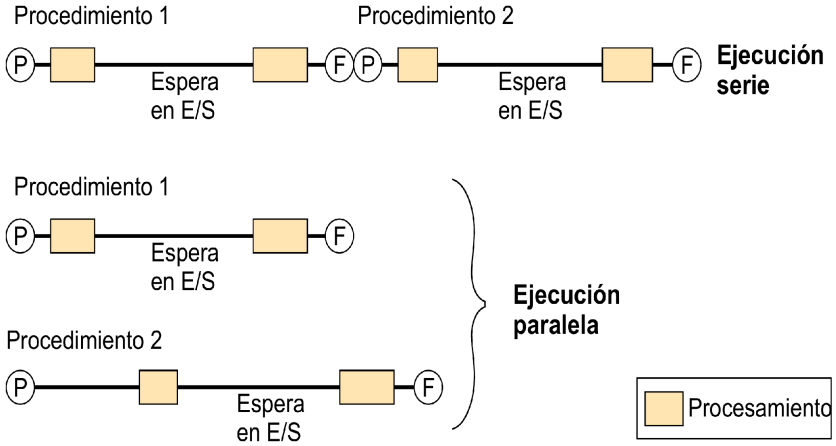
Modelo de proceso monohilo



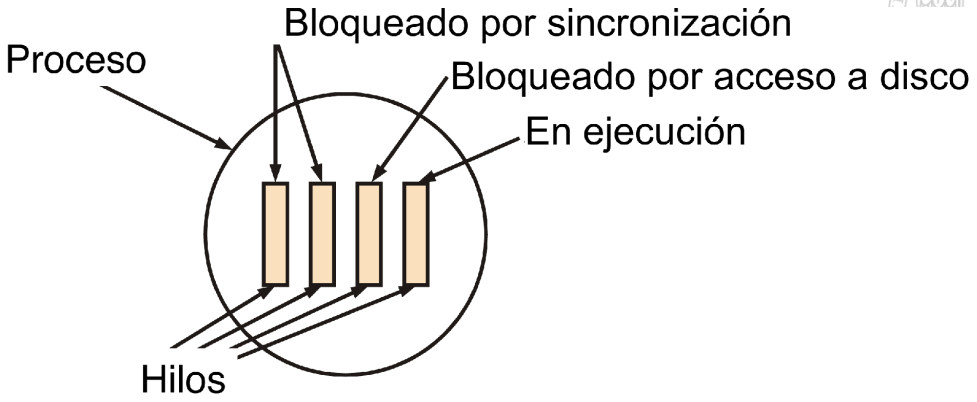
Modelo de proceso multihilo



Paralelización usando hilos



Estados de un hilo



Ventajas threads vs procesos

- Operaciones relacionadas con creación, destrucción, planificación y sincronización son más costosas para procesos que para hilos
  - Hasta un orden de magnitud
- El cambio de contexto entre hilos de un mismo proceso es menos costoso que entre hilos de distintos procesos
  - En el primer caso no es necesario cambiar el espacio de direcciones "activo" de usuario



Diseño con hilos

- Permite separación de tareas
- Permite división de tareas
- Aumenta la velocidad de ejecución del trabajo
- Paralelismo
- Llamadas al sistema bloqueantes
- Programación concurrente (memoria compartida)
  - Variables o estructuras de datos compartidas
  - Funciones reentrantes
    - Imaginar llamadas simultáneas a la misma función (no necesariamente de forma coordinada)
  - Mecanismos de sincronización entre hilos mutexes, semáforos,...
  - Simplicidad vs. exclusión en el acceso

Alternativas al diseño multihilo

- Proceso con un solo hilo
  - No hay paralelismo
    - Llamadas al sistema bloqueantes
  - Paralelismo gestionado por el programador
    - Llamadas al sistema no bloqueantes
- Múltiples procesos convencionales cooperando
  - Permite paralelismo
  - No comparten variables
    - Necesario emplear mecanismo de comunicación entre procesos (proporcionado por el SO)
  - Mayor sobrecarga de ejecución

API de POSIX Threads



- `int pthread_create(pthread_t *thread, const pthread_attr_t *attr,void *(*func)(void *), void *arg)`
  - Crea un hilo que ejecuta func con argumento `arg` y atributos `attr`
    - Los atributos permiten especificar: tamaño de la pila, prioridad, política de planificación, etc.
    - Existen diversas llamadas para modificar los atributos
- `int pthread_join(pthread_t thid, void **value)`
  - Suspende la ejecución de un hilo hasta que termina el hilo con identificador `thid`
    - Devuelve el estado de terminación del hilo
- `int pthread_exit(void *value)`
  - Permite a un hilo finalizar su ejecución, indicando el estado de terminación del mismo
- `pthread_t pthread_self(void)`
  - Devuelve el identificador del thread que ejecuta la llamada

API de POSIX Threads (Cont.)

- `int pthread_attr_setdetachstate(pthread_attr_t *attr, int detachstate)`
  - Establece el estado de terminación de un hilo.
    - Si `detachstate = PTHREAD_CREATE_DETACHED` el hilo liberará sus recursos cuando finalice su ejecución
    - Si `detachstate = PTHREAD_CREATE_JOINABLE` no se liberarán los recursos, es necesario utilizar `pthread_join()`

Ejemplo hilos

threads_example.c

```
#include <stdio.h>
#include <pthread.h>
#define MAX_THREADS 10

void* func(void* arg) {
    printf("Thread %d \n", pthread_self());
    pthread_exit(0);
}

int main(void) {
    int j;
    pthread_attr_t attr;
    pthread_t thid[MAX_THREADS];

    pthread_attr_init(&attr);

    for(j = 0; j < MAX_THREADS; j ++)
        pthread_create(&thid[j], &attr, func, NULL);

    for(j = 0; j < MAX_THREADS; j ++)
        pthread_join(thid[j], NULL);

    return 0;
}
```